

Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

Solution:

```
using System;

class Logger
{
    private static Logger instance;

    private Logger()
    {
    }

    public static Logger GetInstance()
    {
        if (instance == null)
        {
            instance = new Logger();
        }

        return instance;
    }

    public void Log(string message)
    {
        Console.WriteLine("LOG: " + message);
    }
}

class Program
{

```

```
static void Main()
{
    Logger logger1 = Logger.GetInstance();
    Logger logger2 = Logger.GetInstance();

    logger1.Log("Application Started");

    Console.WriteLine(
        "Same Instance: " + Object.ReferenceEquals(logger1, logger2));
}
}
```

Exercise 2: Implementing the Factory Method Pattern

Solution:

```
using System;

interface IDocument
{
    void Open();
}

class WordDocument : IDocument
{
    public void Open()
    {
        Console.WriteLine("Opening Word Document");
    }
}
```

```
class PdfDocument : IDocument
{
    public void Open()
    {
        Console.WriteLine("Opening PDF Document");
    }
}
```

```
class ExcelDocument : IDocument
{
    public void Open()
    {
        Console.WriteLine("Opening Excel Document");
    }
}
```

```
abstract class DocumentFactory
{
    public abstract IDocument CreateDocument();
}
```

```
class WordFactory : DocumentFactory
{
    public override IDocument CreateDocument()
    {
        return new WordDocument();
    }
}
```

```
class PdfFactory : DocumentFactory
{
    public override IDocument CreateDocument()
    {
        return new PdfDocument();
    }
}

class ExcelFactory : DocumentFactory
{
    public override IDocument CreateDocument()
    {
        return new ExcelDocument();
    }
}

class Program
{
    static void Main()
    {
        DocumentFactory factory = new PdfFactory();

        IDocument doc = factory.CreateDocument();

        doc.Open();
    }
}
```

Exercise 3: Implementing the Builder Pattern

Solution:

using System;

class Computer

{

public string CPU { get; }

public int RAM { get; }

public int Storage { get; }

private Computer(Builder builder)

{

CPU = builder.CPU;

RAM = builder.RAM;

Storage = builder.Storage;

}

public void Display()

{

Console.WriteLine("CPU: " + CPU);

Console.WriteLine("RAM: " + RAM + " GB");

Console.WriteLine("Storage: " + Storage + " GB");

}

public class Builder

{

```
public string CPU;

public int RAM;

public int Storage;


public Builder SetCPU(string cpu)
{
    CPU = cpu;
    return this;
}


public Builder SetRAM(int ram)
{
    RAM = ram;
    return this;
}


public Builder SetStorage(int storage)
{
    Storage = storage;
    return this;
}


public Computer Build()
{
    return new Computer(this);
}
}
```

```

class Program
{
    static void Main()
    {
        Computer computer = new Computer.Builder()
            .SetCPU("Intel i7")
            .SetRAM(16)
            .SetStorage(512)
            .Build();

        computer.Display();
    }
}

```

Exercise 4: Implementing the Adapter Pattern

Solution:

```
using System;
```

```

interface IPaymentProcessor
{
    void ProcessPayment(double amount);
}

```

```

class PayPalGateway
{
    public void MakePayment(double amount)
    {
        Console.WriteLine("Paid using PayPal: " + amount);
    }
}

```

```
}
```

```
class StripeGateway
```

```
{
```

```
    public void Pay(double amount)
```

```
    {
```

```
        Console.WriteLine("Paid using Stripe: " + amount);
```

```
    }
```

```
}
```

```
class PayPalAdapter : IPaymentProcessor
```

```
{
```

```
    private PayPalGateway gateway;
```

```
    public PayPalAdapter(PayPalGateway gateway)
```

```
    {
```

```
        this.gateway = gateway;
```

```
    }
```

```
    public void ProcessPayment(double amount)
```

```
    {
```

```
        gateway.MakePayment(amount);
```

```
    }
```

```
}
```

```
class StripeAdapter : IPaymentProcessor
```

```
{
```

```
    private StripeGateway gateway;
```



```

public StripeAdapter(StripeGateway gateway)
{
    this.gateway = gateway;
}

public void ProcessPayment(double amount)
{
    gateway.Pay(amount);
}
}

class Program
{
    static void Main()
    {
        IPaymentProcessor payment =
            new PayPalAdapter(new PayPalGateway());

        payment.ProcessPayment(1000);
    }
}

```

Exercise 5: Implementing the Decorator Pattern

Solution:

```

using System;

interface INotifier
{

```

```

        void Send(string message);
    }

    class EmailNotifier : INotifier
    {
        public void Send(string message)
        {
            Console.WriteLine("Email: " + message);
        }
    }

    abstract class NotifierDecorator : INotifier
    {
        protected INotifier notifier;

        public NotifierDecorator(INotifier notifier)
        {
            this.notifier = notifier;
        }

        public abstract void Send(string message);
    }

    class SMSNotifierDecorator : NotifierDecorator
    {
        public SMSNotifierDecorator(INotifier notifier)
            : base(notifier)
        {
        }
    }

```

```

        public override void Send(string message)
        {
            notifier.Send(message);

            Console.WriteLine("SMS: " + message);
        }
    }

    class SlackNotifierDecorator : NotifierDecorator
    {
        public SlackNotifierDecorator(INotifier notifier)
            : base(notifier)
        {
        }

        public override void Send(string message)
        {
            notifier.Send(message);

            Console.WriteLine("Slack: " + message);
        }
    }

    class Program
    {
        static void Main()
        {
            INotifier notifier =
                new SlackNotifierDecorator(
                    new SMSNotifierDecorator(
                        new EmailNotifier()));

```

```
        notifier.Send("System Alert");  
    }  
}
```

Exercise 6: Implementing the Proxy Pattern

Solution:

```
using System;  
  
interface IImage  
{  
    void Display();  
}  
  
class RealImage : IImage  
{  
    private string fileName;  
  
    public RealImage(string fileName)  
    {  
        this.fileName = fileName;  
        LoadFromServer();  
    }  
  
    private void LoadFromServer()  
    {  
        Console.WriteLine("Loading image from server: " + fileName);  
    }  
}
```

```

    public void Display()
    {
        Console.WriteLine("Displaying image: " + fileName);
    }
}

class ProxyImage : IImage
{
    private string fileName;
    private ReallImage reallImage;
    public ProxyImage(string fileName)
    {
        this.fileName = fileName;
    }

    public void Display()
    {
        if (reallImage == null)
        {
            reallImage = new ReallImage(fileName);
        }
        reallImage.Display();
    }
}

class Program
{
    static void Main()
    {
        IImage image = new ProxyImage("sample.jpg");
    }
}

```

```
        image.Display();  
        Console.WriteLine();  
  
        image.Display();  
    }  
}
```

Exercise 7: Implementing the Observer Pattern

Solution:

```
using System;  
using System.Collections.Generic;  
  
interface IObservable  
{  
    void Update(double price);  
}  
  
interface IStock  
{  
    void RegisterObserver(IObservable observer);  
    void RemoveObserver(IObservable observer);  
    void NotifyObservers();  
}  
  
class StockMarket : IStock  
{  
    private List<IObservable> observers = new List<IObservable>();
```

```

private double stockPrice;

public void RegisterObserver(IObserver observer)
{
    observers.Add(observer);
}

public void RemoveObserver(IObserver observer)
{
    observers.Remove(observer);
}

public void NotifyObservers()
{
    foreach (IObserver observer in observers)
    {
        observer.Update(stockPrice);
    }
}

public void SetPrice(double price)
{
    stockPrice = price;
    NotifyObservers();
}
}

class MobileApp : IObserver
{

```

```
public void Update(double price)
{
    Console.WriteLine("Mobile App: Stock Price Updated = " + price);
}
}
```

```
class WebApp : IObservable
{
    public void Update(double price)
    {
        Console.WriteLine("Web App: Stock Price Updated = " + price);
    }
}
```

```
class Program
{
    static void Main()
    {
        StockMarket market = new StockMarket();

        market.RegisterObserver(new MobileApp());
        market.RegisterObserver(new WebApp());

        market.SetPrice(1500);
        market.SetPrice(1800);
    }
}
```


Exercise 8: Implementing the Strategy Pattern

Solution:

using System;

interface IPaymentStrategy

```
{  
    void Pay(double amount);  
}
```

class CreditCardPayment : IPaymentStrategy

```
{  
    public void Pay(double amount)  
    {  
        Console.WriteLine("Paid " + amount + " using Credit Card");  
    }  
}
```

class PayPalPayment : IPaymentStrategy

```
{  
    public void Pay(double amount)  
    {  
        Console.WriteLine("Paid " + amount + " using PayPal");  
    }  
}
```

class PaymentContext

```
{
```

```

private IPaymentStrategy strategy;

public void SetStrategy(IPaymentStrategy strategy)
{
    this.strategy = strategy;
}

public void ExecutePayment(double amount)
{
    strategy.Pay(amount);
}
}

class Program
{
    static void Main()
    {
        PaymentContext context = new PaymentContext();
        context.SetStrategy(new CreditCardPayment());
        context.ExecutePayment(5000);
        context.SetStrategy(new PayPalPayment());
        context.ExecutePayment(3000);
    }
}

```

Exercise 9: Implementing the Command Pattern

Solution:

```

using System;

interface ICommand
{

```

```

        void Execute();
    }

    class Light
    {
        public void TurnOn()
        {
            Console.WriteLine("Light ON");
        }
        public void TurnOff()
        {
            Console.WriteLine("Light OFF");
        }
    }
    class LightOnCommand : ICommand
    {
        private Light light;

        public LightOnCommand(Light light)
        {
            this.light = light;
        }
        public void Execute()
        {
            light.TurnOn();
        }
    }

    class LightOffCommand : ICommand

```

```

{
    private Light light;
    public LightOffCommand(Light light)
    {
        this.light = light;
    }
    public void Execute()
    {
        light.TurnOff();
    }
}
class RemoteControl
{
    private ICommand command;
    public void SetCommand(ICommand command)
    {
        this.command = command;
    }
    public void PressButton()
    {
        command.Execute();
    }
}

class Program
{
    static void Main()
    {
        Light light = new Light();
    }
}

```

```

        RemoteControl remote = new RemoteControl();

        remote.SetCommand(new LightOnCommand(light));
        remote.PressButton();

        remote.SetCommand(new LightOffCommand(light));
        remote.PressButton();
    }
}

```

Exercise 10: Implementing the MVC Pattern

Solution:

```
using System;
```

```

class Student
{
    public string Name { get; set; }
    public int Id { get; set; }
    public string Grade { get; set; }

    public Student(string name, int id, string grade)
    {
        Name = name;
        Id = id;
        Grade = grade;
    }
}

```

```
class StudentView
```

```

{
    public void DisplayStudentDetails(string name, int id, string grade)
    {
        Console.WriteLine("Student Details");
        Console.WriteLine("-----");
        Console.WriteLine("Name : " + name);
        Console.WriteLine("ID  : " + id);
        Console.WriteLine("Grade : " + grade);
    }
}

```

```

class StudentController
{
    private Student model;
    private StudentView view;
    public StudentController(Student model, StudentView view)
    {
        this.model = model;
        this.view = view;
    }
    public void SetStudentName(string name)
    {
        model.Name = name;
    }

    public void SetStudentGrade(string grade)
    {
        model.Grade = grade;
    }
}

```

```

public void UpdateView()
{
    view.DisplayStudentDetails(
        model.Name,
        model.Id,
        model.Grade
    );
}
}

class Program
{
    static void Main()
    {
        Student student = new Student("John", 101, "A");

        StudentView view = new StudentView();

        StudentController controller =
            new StudentController(student, view);

        controller.UpdateView();

        Console.WriteLine();

        controller.SetStudentName("Alex");
        controller.SetStudentGrade("A+");
    }
}

```

```
        controller.UpdateView();  
    }  
}
```

Exercise 11: Implementing Dependency Injection

Solution:

```
using System;
```

```
interface ICustomerRepository  
{  
    string FindCustomerById(int id);  
}
```

```
class CustomerRepositoryImpl : ICustomerRepository  
{  
    public string FindCustomerById(int id)  
    {  
        return "Customer Found. ID = " + id;  
    }  
}
```

```
class CustomerService  
{  
    private readonly ICustomerRepository repository;  
  
    public CustomerService(ICustomerRepository repository)  
    {  
        this.repository = repository;  
    }  
}
```



```
}

public void GetCustomer(int id)
{
    Console.WriteLine(repository.FindCustomerById(id));
}
}
```

```
class Program
{
    static void Main()
    {
        ICustomerRepository repository =
            new CustomerRepositoryImpl();

        CustomerService service =
            new CustomerService(repository);

        service.GetCustomer(101);
    }
}
```